

#block of code can perform a specific task:

"""

```
def func_name():
    print("yo")
    func_name()
    func_name()          # code is reusable whenever we want
```

"""

function -- Eg: name == parameter and "Zoro" == Argument

"""

```
def func(name):
    print(f"Yo {name}")
    func("Zoro")
    func("Sanji")
```

"""

function odd or even

"""

```
def odd_even(n):
    if n%2 == 0:
        print(f"{n} is even")
    else:
        print(f"{n} is odd")
```

odd_even(5)

odd_even(6)

"""

function inside get input

"""

```
def func(name):
    print(f"yo {name}")
    n=input("Enter a name: ")
    func(n)
```

m=input("Enter a name: ")

func(m)

"""

#

Types of Arguments

1. Positional

2. Keyword

3. Default

4. Arbitrary Positional

5. Arbitrary Keyword

#

1. Positional Argument: -- {ordered format reading} -- {also all arguments}

"""

```
def pos_arg(name):
    print(f" yo {name}")
    pos_arg("Sanji")
"""

def pos_two(name,reg):
    print(f"yo {name} reg:{reg}")
    #pos_two("luffy")          # -- TypeError: pos_two() missing 1 required positional argument: 'reg'
    pos_two("zoro","hell")
    pos_two("hell","zoro")      # -- here order changes
"""
```

2. Keyword Argument:

```
"""
def key_arg(name,no):
    print(f" yo {name} no: {no}")
    key_arg(name="Ace",no=2)
"""

# keyword argument follows positional argument
"""
def key_two(name,title,fruit):
    print(f"{name} {title} {fruit}")

# keyword argument follows positional argument
key_two("ace",title="2nd div captain",fruit="fire fire fruit") # this works
"""

# but positional argument does not follows keyword argument

#key_two(name="ace","2nd div captain","fire fire fruit") # Syntax error
"""
```

3. Default Argument:

```
"""
# -- default parameter doesn't follows without default parameter : -- syntax error
"""

def def_arg(a=1,b):      # syntax error
    print(a+b)
    def_arg(b=2)
"""

def default_arg(a=0,b=1):
    print(a+b)
    default_arg(10,5)
    default_arg()        # -- here take default values what we give
"""
```

4. Arbitrary positional Argument: * -- means multiple positional arguments can read
-- tuple of arguments

```
"""
def arb_pos_arg(*args):
    print(sum(args))
"""
```

```
arb_pos_arg(10)
arb_pos_arg(10,20)
arb_pos_arg(10,20,30)
arb_pos_arg(10,20,30,40,50,60,70)
'''
```

```
'''
def arb_pos_two(*args):
s=0
for i in args:
s+=i
print(s)
arb_pos_two(10,20,30,40,50,60)
'''
```

```
# 5. Arbitrary Keyword Argument: ** -- means multiple keyword arguments can read
# dictionary of arguments
'''
```

```
def arb_key_arg(**kwargs):
print(kwargs)
arb_key_arg(name=ace,no=2)
'''
```

```
'''
def arb_key_two(**kwargs):
for i,j in kwargs.items():
print(i,j)
arb_key_two(name="ace",title="2nd div captain",fruit="fire fire fruit")
'''
```

```
#
```

```
# Built-in functions:
```

Date: 03-03-2026

```
## abs (absolute):
'''
```

```
a=-3
b=3
print(abs(a),abs(b))
'''
```

```
## pow (power):
'''
```

```
p=5
print(pow(p,2))
'''
```

```
## sum (addition):
'''
```

```
s=[10,20,30]
su=(10,20,30)
print(sum(s),sum(su))
'''
```

```
## range (range of int) :
```

```

"""
r=7
print(range(r))
"""

## min    (minimum num in group of nums):

##m=[10,20,30,40,50]

##print(min(m))

## max    (maximum num in group of nums):

##print(max(m))

## round  (round of a float into int):
"""
a=6.5
b=6.6
print(round(a),round(b))
"""

"""
a=[False,False]
b=[True,False]
c=[False,True]
d=[True,True]

e=[1]
f=[0]

g=[""]
h=[" "]
i=[]
j=""
"""

## any    -- OR GATE -- (any one True --op will True):
"""
print(any(a))
print(any(b))
print(any(c))
print(any(d))
print("-----")
print(any(e))
print(any(f))
print("-----")
print(any(g))
print(any(h))
print("-----")
print(any(i))
print(any(j))
"""

## all    -- AND GATE -- (all every one have to be True for -- op True)
"""

```

```

print(all(a))
print(all(b))
print(all(c))
print(all(d))
print("-----")
print(all(e))
print(all(f))
print("-----")
print(all(g))
print(all(h))
print("-----")
print(all(i))
print(all(j))
"""

```

```

## bin
"""

```

```

a=1
print(bin(a))
"""

```

```

## boolean
"""

```

```

m=0
n=1
o=""
p=" "
q=[]
r=[" "]
s=None

```

```

print(bool(m))
print(bool(n))
print(bool(o))
print(bool(p))
print(bool(q))
print(bool(r))
print(bool(s))
"""

```

```

## enumerate()
"""

```

```

e=['ace','sabo','luffy']
for i,j in enumerate(e):
    print(i,j)

```

```

for i,j in enumerate(e,7):          # --- where will start
    print(i,j)
"""

```

```

## zip()
"""

```

```

a=[10,20,30]
b=[30,40,50]
d=[1,2,3,4]

```

```
c=zip(a,b)
print(list(c))
```

```
e=zip(a,d)
print(list(e))
```

```
"""
## unzip --- a.b =zip(*x)
"""
```

```
f=[(10, 30), (20, 40), (30, 50)]
g,h=zip(*f)
print(g,h)
```

```
"""
##i=dict [g] [h]
##print(i)
"""
```

```
## shallow copy
```

```
# 1. shallow copy creates a new outer object
# but references the same nested mutable objects.
```

```
# 2. if nested mutable objects mutated, it will affect original.
```

```
import copy
```

```
"""
a=[1,2,3,4]
b=copy.copy(a)
print(a,id(a))
print(b,id(b))
"""
```

```
"""
c=[[1,2],[3,4],5]
d=copy.copy(c)
```

```
print(c,id(c))
print(d,id(d))
d.append(6)          # it doesn't affects c
d[0].append(0)      ##### --- it does affects c ---
print(c,id(c))
print(d,id(d))
"""
```

```
## deep copy
```

```
# 1. deep copy recursively copies all nested objects
# creating independent memory allocations.
```

```
"""
a=[1,2,3,4]
b=copy.deepcopy(a)
print(a,id(a))
print(b,id(b))
"""
```

```

"""

c=[[1,2],[3,4],5]
d=copy.deepcopy(c)

print(c,id(c))
print(d,id(d))
d.append(6)      # it doesn't affects c
d[0].append(0)   # it doesn't affects c
print(c,id(c))
print(d,id(d))
"""

## return      --- (Back to the caller Doesn't read any function code after return)

```

```

"""
def return_2():
    return "Ace"      # -- exit here
    return "sabo"     # -- doesn't read here
    print(return_2())

```

```

def return_2():
    return "Ace"      # -- exit here
    print("sabo")     # -- doesn't read here
    print(return_2())

```

```

def return_2():
    print("Ace" )     # -- exit here
    return "sabo"     # -- does read here
    print(return_2())
"""

```

#

recursive function

Date: 04-03-2026

is a function That calls itself repeatedly until the condition is satisfied.

Factorial

```

"""
def fact(n):
    if n==0:
        return 1
    else:
        return n*fact(n-1)
    print(fact(5))
"""

```

```

# sum of n numbers
"""
def sum(n):
if n==0:
return 0
else:
return n+sum(n-1)
print(sum(5))
"""

# Fibonacci series:
"""
def fib(n):
if n<=1:
return n
else:
return fib(n-2)+fib(n-1)
num=int(input())
for i in range(num):
print(fib(i))
"""

# string reverse using recursive:
"""
def rev(n):
if len(n)<=1:
return n
else:
return rev(n[1:])+n[0]
print(rev("python"))
"""

# Palindrome Using recursive
"""
def palin(n):
if len(n)<=1:
return n
else:
return palin(n[1:])+n[0]
n=input()
if n==palin(n):
print("Palindrome")
else:
print("Not Palindrome")
"""

# list number reverse:
"""
def rev(n):
if len(n)==0:
return []
else:
return rev(n[1:])+[n[0]]
print(rev([1,2,3,4]))

#n=[0]
#print(n[1:])
"""

```


#

lambda function

Date: 05-03-2026

"""

Its a anonymous function
Its a one line function
No need to use def keyword

"""

#syntax:

lambda arguments:expression

using print

"""

```
def func():  
    print('yo')  
func()
```

using lambda

```
func = lambda: print('yo')  
func()
```

"""

using return

"""

```
def func():  
    return('Yo')  
print(func())
```

#

```
func = lambda : 'Yo'  
print(func())
```

"""

argument passing

using print

"""

```
def add (a,b):  
    print(a+b)  
add(10,8)
```

#

```
add = lambda a,b : print(a+b)  
add(8,10)
```

"""

##using return

"""

```
def add(a,b):  
    return a+b
```

```
print(add(10,5))
```

```
#
```

```
add = lambda a,b : a+b
```

```
print(add(6,7))
```

```
'''
```

```
# lamda function using if ---- odd or even
```

```
# using print
```

```
'''
```

```
def oddEven(a):
```

```
if a%2==0:
```

```
print("even")
```

```
else:
```

```
print("odd")
```

```
oddEven(1)
```

```
'''
```

```
#
```

```
'''
```

```
n=lambda a : print("even") if a%2==0 else print("odd")
```

```
n(6)
```

```
'''
```

```
# using return
```

```
'''
```

```
def odd(a):
```

```
if a%2==0:
```

```
return even
```

```
return odd
```

```
print(4)
```

```
'''
```

```
#
```

```
'''
```

```
m=lambda a: 'even' if a%2==0 else 'odd'
```

```
print(m(6))
```

```
'''
```

```
# sort and reverse:
```

```
'''
```

```
a=[9,4,7,2,6]
```

```
a.sort(reverse=True)
```

```
#a.sort()
```

```
print(a)
```

```
'''
```

```
# sort key = len means which is small count based sorting
```

```
'''
```

```
a=['ace','luffy','sabo']
```

```
a.sort(key=len)
```

```
print(a)
```

```
'''
```

```
# value based ascending order sorting
```

```
'''
```

```
a=[('luffy',3),('sabo',2),('ace',1)]
```

```

a.sort( key = lambda z:z[1])
print(a)
"""

#
"""

a=[('luffy',3),('sabo',2),('ace',1)]
a.sort( key = lambda z:z[1],reverse = True)
print(a)
"""

# sorted -- which is not affecting original
"""

a=['ace','luffy','sabo']
b=sorted(a)          # -- alphabetic order sorted
print(a)
print(b)
"""

#
"""

a=['ace','luffy','sabo']
b=sorted(a,key=len) # -- len value based sorted
print(a,b)
"""

#      -- value based
"""

a={'luffy':3,'sabo':4,'ace':1}
b=dict(sorted (a.items(),key=lambda z:z[1]))
print(a,b)
"""

# count words:

##a="I learn python"
"""

count = 1
for i in range(len(a)):
if a[i] == " ":
count+=1
print(count)
"""

# function
"""

def bit(a):
return len(a.split())
print(bit(a))
"""

# lambda function
"""

c= lambda a:len(a.split())
print(c(a))
"""

#

```

```
## Higher order function
```

```
## map
```

```
## syntax
```

```
##          map(function,iterables)
```

```
## definition -- Applies a fuction to the elements of an iterables
```

```
##a=[1,2,3,4,5]
```

```
##b=[]
```

```
# normal function:
```

```
'''
```

```
def func(x):
```

```
    return x*2
```

```
for i in a:
```

```
    b.append(func(i))
```

```
print(b)
```

```
'''
```

```
# lambda function
```

```
'''
```

```
c= lambda x: x*2
```

```
for i in a:
```

```
    b.append(c(i))
```

```
print(b)
```

```
'''
```

```
# map and lambda
```

```
'''
```

```
b=map(lambda x:x*2,a)  ##  x:x*2 is function and a is iterables
```

```
print(list(b))
```

```
'''
```

```
# def and map()
```

```
'''
```

```
def func(x):
```

```
    return x*2
```

```
b=map (func , a)
```

```
print(list(b))
```

```
'''
```

```
# len of str in list of elements:
```

```
# def and map
```

```
'''
```

```
a=['ace','luffy','sabo']
```

```
def func(x):
```

```
    return len(x)
```

```
b=map(func,a)
```

```
print(list(b))
```

```
'''
```

```
# map and lambda
```

```

'''
a=['ace','luffy','sabo']
b=map(lambda x:len(x),a)
print(list(b))
'''

#-----

# str into int
'''
a=['1','4','3','2','5']
b = map (int,a)
print(list(b))
'''

# filter
# syntax

#          filter(function,iterables)

# A function is used to filter the element of an iterable

# def and map

'''
def func(x):
return x%2==0
a=[1,2,3,4,5,6]
b=map(func,a)
print(list(b))
'''

# def and filter
'''
def func(x):
return x%2==0
a=[1,2,3,4,5,6]
b=filter(func,a)
print(list(b))
'''

# filter and lambda
'''
a=[1,2,3,4,5,6]
b=filter(lambda x:x%2==0 , a)
print(list(b))
'''

#-----

# filter and def

'''
def func(x):
return len(x)==4
b=filter(func,a)
a=['ace','luffy','sabo','zoro']

```

```
print(list(b))
```

```
"""
```

```
# filter and lambda
```

```
"""
```

```
a=['ace','luffy','sabo','zoro']
```

```
b=filter(lambda x:len(x)==4,a)
```

```
print(list(b))
```

```
"""
```

```
#
```

```
# Higher order function
```

Date: 06-03-2026

```
# filter and lambda
```

```
"""
```

```
a=['sabo','zoro','ace','sanji','str','sky','aei']
```

```
c=filter(lambda y:y.endswith('o'),a)
```

```
print(list(c))
```

```
"""
```

```
#filter and lambda
```

```
"""
```

```
a=['sabo','zoro','ace','sanji','str','sky','aei']
```

```
d=filter(lambda x:x.startswith('s'),a)
```

```
print(list(d))
```

```
"""
```

```
# filter and lambda and for and if:    any vowels
```

```
"""
```

```
a=['sabo','zoro','ace','sanji','str','sky','aei']
```

```
e=filter(lambda x:any(i in ('a','e','i','o','u') for i in x.lower()),a)
```

```
print(list(e))
```

```
"""
```

```
# filter and lambda and for and if:    all vowels
```

```
"""
```

```
a=['sabo','zoro','ace','sanji','str','sky','aei']
```

```
f=filter(lambda x:all(i in ('a','e','i','o','u') for i in x.lower()),a)
```

```
print(list(f))
```

```
"""
```

```
#_____
```

```
# syntax:    reduce (function,iterable)
```

```
## A function is used to reduce a list to a {--single--} value
```

```
# add
```

```
"""
```

```
from functools import reduce
```

```
a=[1,2,3,4,5]
b=reduce (lambda x,y:x+y,a)
print(b)
'''
```

```
# multiply
'''
```

```
from functools import reduce
```

```
a=[1,2,3,4,5]
b=reduce (lambda x,y:x*y,a)
print(b)
'''
```

```
# find the largest number in the list
'''
```

```
from functools import reduce
a=[10,20,30,40,50]
b=reduce(lambda x,y:x if x>y else y,a)
print(b)
'''
```

```
# Smallest Number
'''
```

```
from functools import reduce
m=['ace','sabo','luffy']
c=reduce (lambda x,y:x if x<y else y,a)
print(c)
'''
```

```
# max len of str
'''
```

```
from functools import reduce
m=['ace','sabo','luffy']
o=reduce(lambda x,y:y if len(x) < len(y) else y,m)
print(o)
'''
```

```
# min len of str
'''
```

```
from functools import reduce
```

```
a=[[1,2],[3,4],[5,6]]
n=reduce(lambda x,y:y if len(x)>len(y) else x,m)
print(n)
'''
```

```
# nested list into list
'''
```

```
from functools import reduce
```

```
a=[[1,2],[3,4],[5,6]]
b=reduce(lambda x,y:x+y,a )      # -- here op is single list
print(b)
'''
```

```
from functools import reduce
c="i learn python"
```

```
d=reduce (lambda x,y:y+" "+x ,c.split())
print(d)
```

```
"""
# list + string rev lines not
"""
```

```
from functools import reduce
a=['i','learn','python']
b=reduce (lambda x,y: y + " " + x,a)
print(b)
"""
```

```
#-----
```

```
# closer Function:    --- function inside function
```

```
# 1. print + function2() + function1()
```

```
"""
def outer():
def inner():
print("Yo")
inner()
outer()
"""
```

```
# 2. return + print(function2()) + function1()
```

```
"""
def outer():
def inner():
return 'Yowaimo'
print(inner())
outer()
"""
```

```
# 3. return + return func 2() + print(func 1())
```

```
"""
def outer():
def inner():
return 'Daijobu'
return inner()
print(outer())
"""
```

```
#4. return + return Function 2 + print(func1())
```

```
"""
def outer():
def inner():
return 'joy boy'
return inner
print(outer())
"""
```

```
# 5. print + return func 2() + func 1() or print + return func 2 + func 1()
```

```
"""
def outer():
def inner():
```



```
print('Bankai')
return inner()
outer()
"""
# 6. return + return function 2 + a=function 1() + print(a)
"""
```

```
def outer():
def inner():
return 'Gear 5'
return inner()
a=outer()
print(a)
"""
```

```
# _____
```

```
# with argument passing:
```

```
# outer function argument:
"""
```

```
def outer(a):
def inner():
print(a)
inner()
outer('Sun God')
"""
```

```
# inner function argument:
"""
```

```
def outer(a):
def inner(b):
print(a+b)
return inner
outer(10)(8)
"""
```

```
# Scope of variables:
```

```
## -- Local scope
## -- Global scope
## -- Enclosing scope
## -- Built-in Function
#
```

#
